

Backend Development

Topic: AI Study Buddy

Name: C.Dayan Begam

S.Loga Rubika

I.Lubna

Register Number: 212402919

212402942

212402943

Team members NM Id:

C.Dayan Begam NM id: 57E2E9AE21093D5BE054A96F0B9739A3

S.Logarubika NM id: 6D8BB4018C0BE36B0EB49A0835680DD6

I.Lubna NM id: 52DF363A11EAC5B95A34FB215E7EEC98

TABLE OF CONTENT

Date	Contents	Page.No
1.	Introduction	3
2.	Project Overview	3
3.	Architecture	3
4.	Setup instructions	3
5.	Component documentation	4
6.	Stage Management	4
7.	User Interface	5
8.	Styling	5
9.	Testing	5
10.	Screenshots and Demo	6
11.	Known issues	9
12.	Future Enhancements	9

1.Introduction

AI Study Buddy is an intelligent learning application designed to assist students in their academic activities. The main purpose of the project is to provide students with a simple platform where they can access study-related assistance and manage their learning activities.

The application combines a user-friendly interface with a backend system that manages users, authentication, and application data. It is designed to make studying more organized and convenient while providing a foundation for adding AI-powered educational features.

2.Project Overview

AI Study Buddy is a web-based application developed mainly for students. It provides a structured environment for accessing study assistance and managing user accounts.

The system supports different types of users, such as students and administrators. Students can register and log in to the application, while administrators can manage registered users. The backend handles authentication, user information, and communication with the database.

The main objectives of the project are to provide an easy-to-use study platform, maintain secure user access, manage user information efficiently, and create a base for future AI-based learning features.

3. Architecture

The AI Study Buddy follows a client-server architecture. The frontend acts as the user interface through which students interact with the application. The backend processes user requests and provides the required services.

The backend is connected to a MongoDB database, which stores important application information such as user details. Authentication is handled using tokens so that protected features can only be accessed by authorized users.

The overall flow of the system can be represented as:

User → Frontend → Backend API → Database → Backend Response → User

This structure separates the user interface, application logic, and data storage, making the project easier to maintain and develop.

4. Setup Instructions

The project requires a suitable development environment with Node.js, MongoDB, and Visual Studio Code. The required project dependencies are installed before running the application.

The database connection and other important configuration details are maintained through environment variables. This helps keep sensitive information such as database credentials and secret keys separate from the main project code.

After configuring the required settings, the backend server is started locally. The application can then be tested by accessing its available features and APIs through the development environment.

5. Component Documentation

The project consists of several important components that work together to provide the required functionality.

Authentication Component

The authentication component manages user registration and login. During registration, users provide information such as their name, email, password, and role. After successful registration, the system creates the user account.

The login component verifies the user's credentials and provides authentication tokens when the login is successful.

User Management Component

The user management component handles information related to registered users. It allows authorized administrators to view user details and manage user accounts.

Admin Component

The admin component provides additional privileges to administrators. Based on the testing performed, administrators can retrieve the list of users and delete selected user accounts.

Database Component

The database component is responsible for storing and retrieving application information. MongoDB is used to maintain user records and other required data.

6. Stage Management

The development of AI Study Buddy can be divided into different stages.

Planning: The project idea, objectives, and major features were identified.

Requirement Analysis: The required user, authentication, database, and administrative functions were determined.

Design: The application structure and interaction between the frontend, backend, and database were planned.

Development: The required application components and backend services were implemented.

Database Integration: MongoDB was connected to store and manage application data.

Authentication: Registration, login, and role-based access were implemented.

Testing: The different backend operations were tested using Thunder Client to check whether the expected responses were received.

Final Development: The tested components are integrated to form the complete application.

7. User Interface

The User Interface of AI Study Buddy is designed to provide students with a simple and convenient way to interact with the application.

The interface includes important sections such as registration, login, and the main study area. Different options can be provided based on the user's role. Students can access learning features, while administrators can access user management functions.

The interface focuses on clear navigation, readable content, and simple controls so that users can understand and use the application without difficulty.

8. Styling

Styling plays an important role in making AI Study Buddy visually appealing and easy to use. The interface uses consistent layouts, spacing, fonts, buttons, and input fields to create a uniform appearance throughout the application.

The design focuses on simplicity rather than unnecessary visual elements. Important actions such as login, registration, and navigation are presented clearly.

The application can also be designed to be responsive so that its interface remains usable on different screen sizes such as laptops, tablets, and mobile devices.

9. Testing

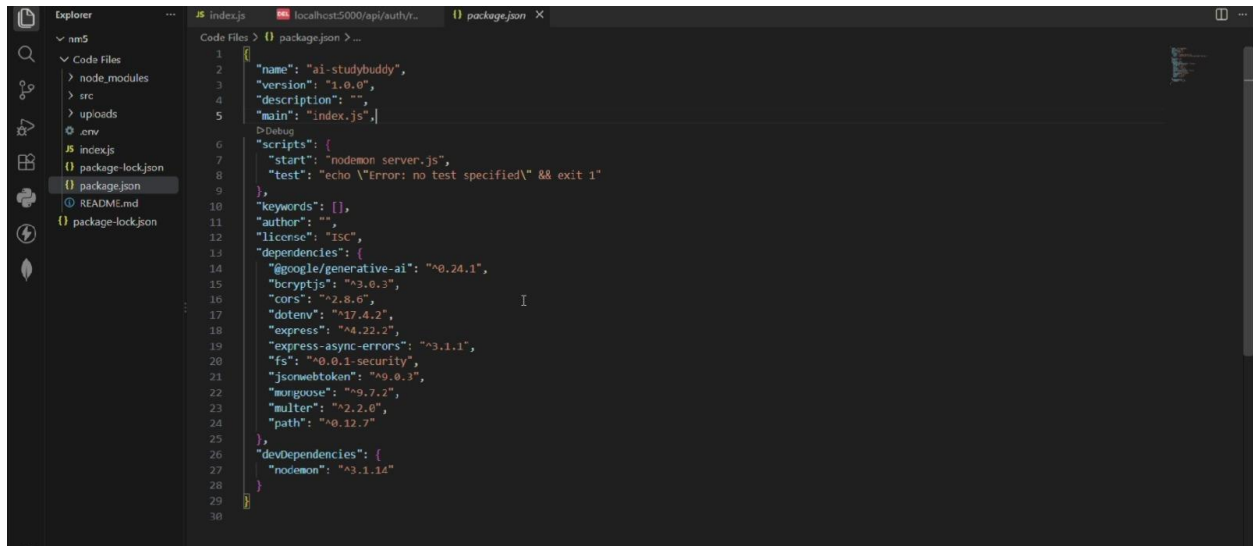
Testing was carried out to verify whether the major backend functions of AI Study Buddy were working correctly. Thunder Client was used to send requests to the application's API endpoints and observe their responses.

The registration function was tested by creating a new student account, and the system returned a successful registration response. The login function was also tested using the registered user's credentials and returned a successful login response.

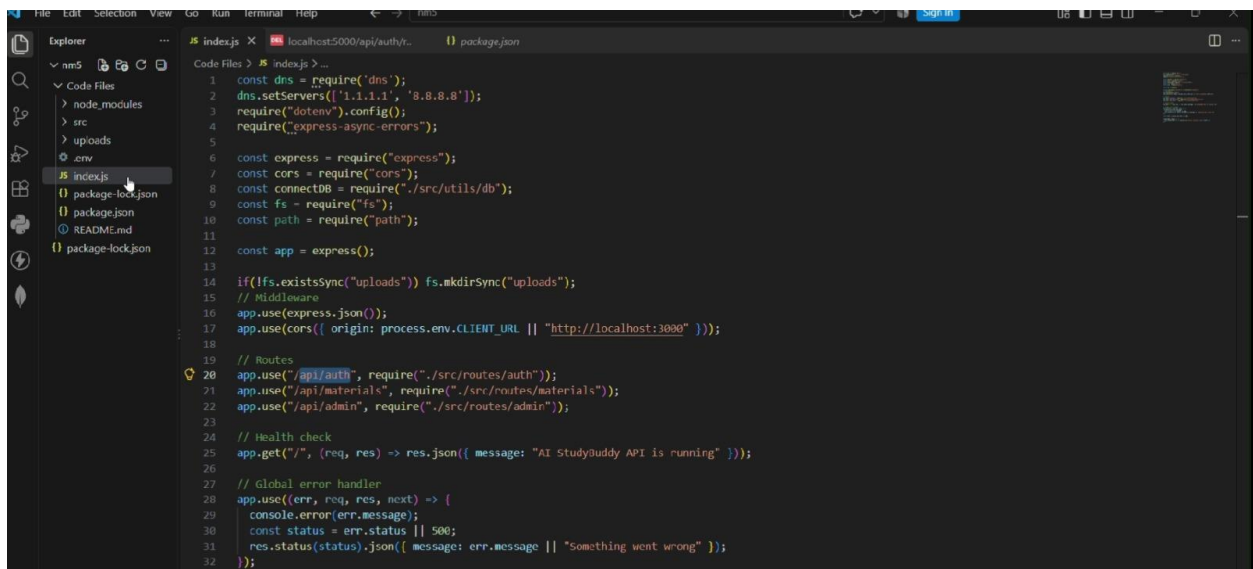
The admin user management functions were tested by retrieving the registered users and performing a user deletion operation. Both operations returned successful responses.

The testing confirms that the major authentication and user management functions demonstrated in the project are working as expected. Further testing can be performed for incorrect passwords, duplicate accounts, unauthorized access, and invalid requests.

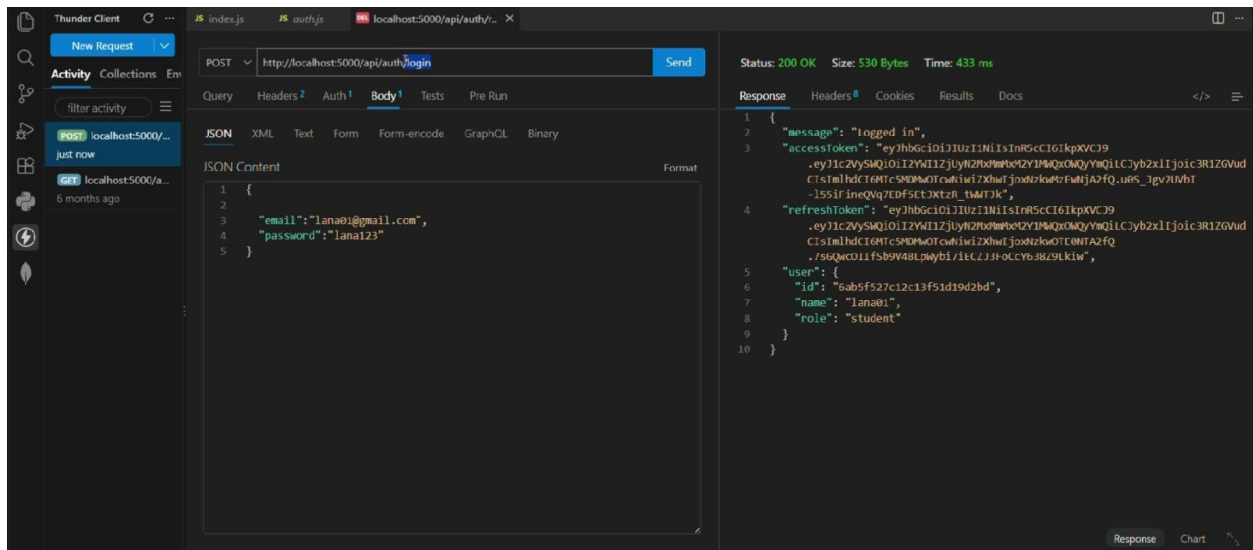
10. Screenshots

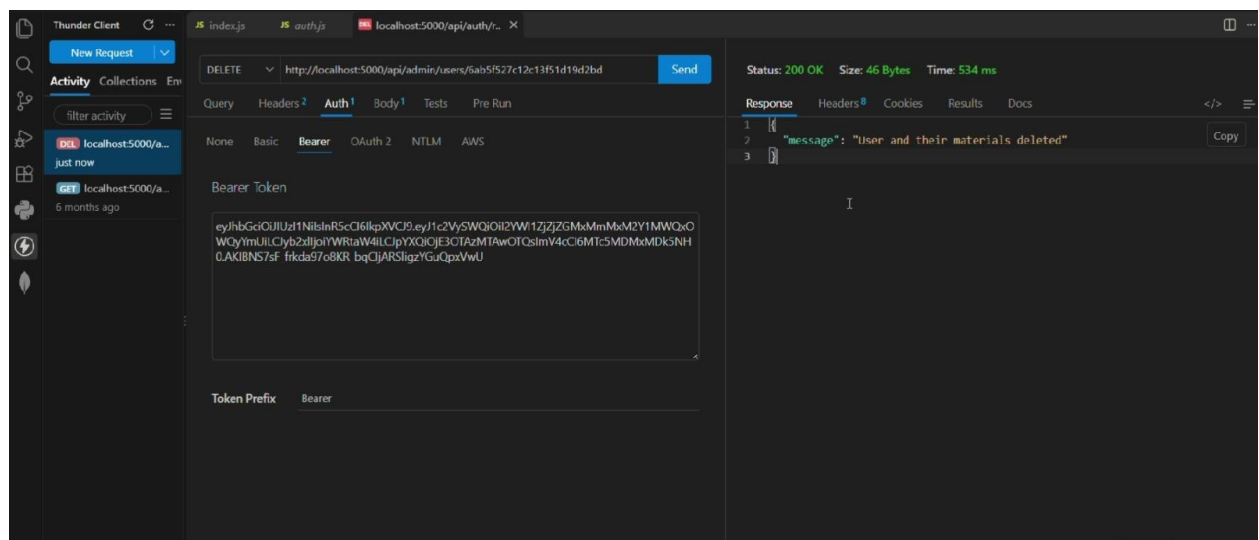
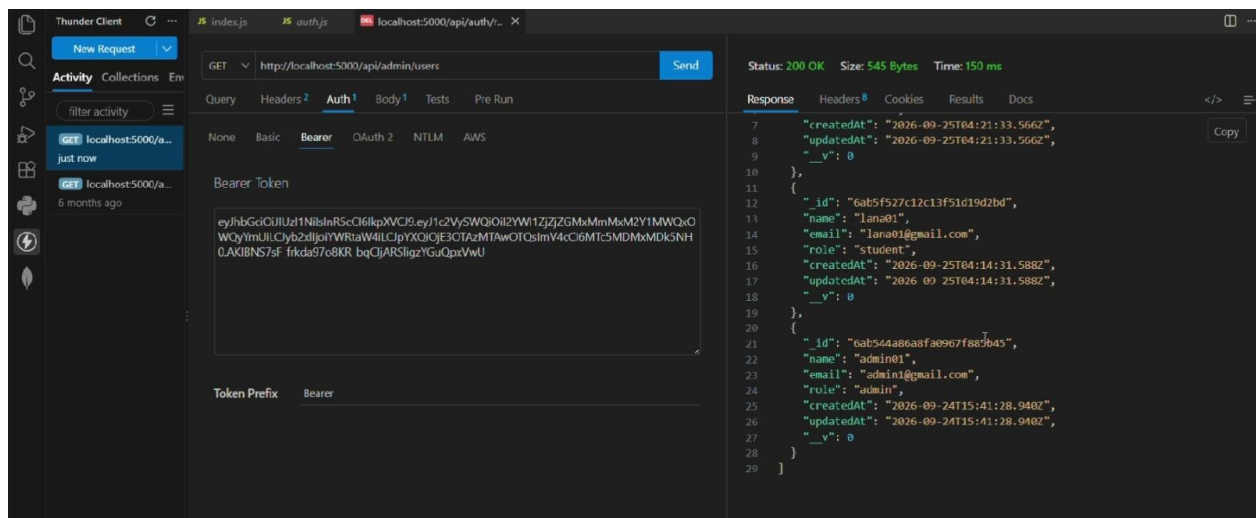


```
1 {
2   "name": "ai-studybuddy",
3   "version": "1.0.0",
4   "description": "",
5   "main": "index.js",
6
7   "scripts": {
8     "start": "nodemon server.js",
9     "test": "echo \\\"error: no test specified\\\" && exit 1"
10  },
11  "keywords": [],
12  "author": "",
13  "license": "ISC",
14  "dependencies": {
15    "@google/generative-ai": "^0.24.1",
16    "bcryptjs": "^3.0.3",
17    "cors": "^2.8.6",
18    "dotenv": "^17.4.2",
19    "express": "^4.22.2",
20    "express-async-errors": "^3.1.1",
21    "fs": "^0.0.1-security",
22    "jsonwebtoken": "^9.0.3",
23    "mongoose": "^9.7.2",
24    "multer": "^2.2.0",
25    "path": "^0.12.7"
26  },
27  "devDependencies": {
28    "nodemon": "^3.1.14"
29  }
30 }
```



```
1 const dns = require("dns");
2 dns.setServers(["1.1.1.1", "8.8.8.8"]);
3 require("dotenv").config();
4 require("express-async-errors");
5
6 const express = require("express");
7 const cors = require("cors");
8 const connectDB = require("../src/utlis/db");
9 const fs = require("fs");
10 const path = require("path");
11
12 const app = express();
13
14 if(!fs.existsSync("uploads")) fs.mkdirSync("uploads");
15 // Middleware
16 app.use(express.json());
17 app.use(cors({ origin: process.env.CLIENT_URL || "http://localhost:3000" }));
18
19 // Routes
20 app.use("/api/auth", require("../src/routes/auth"));
21 app.use("/api/materials", require("../src/routes/materials"));
22 app.use("/api/admin", require("../src/routes/admin"));
23
24 // Health check
25 app.get("/", (req, res) => res.json({ message: "AI Studybuddy API is running" }));
26
27 // Global error handler
28 app.use((err, req, res, next) => {
29   console.error(err.message);
30   const status = err.status || 500;
31   res.status(status).json({ message: err.message || "Something went wrong" });
32 });
```





Demo Video

The main objective of this project is to develop a backend system for an AI Study Buddy that helps students manage their study materials and improve their learning process. The system allows users

to register and log in, upload and manage study materials, and access their learning content securely.

The backend processes user requests, communicates with the database, and provides AI-powered features using Gemini, such as summarizing study materials, generating flashcards, creating quizzes, and preparing study plans.

Through this system, students can easily organize their study materials and use AI-based tools to make their learning more effective. The backend also provides authentication and admin functionalities to ensure that the application is secure and properly managed.

Demo Video Link:

https://drive.google.com/file/d/1-QjXBCoRUKSznI1TJfQ-GeX4qlJe_X1h/view?usp=drivesdk

11. Known Issues

Although the main functionalities have been implemented and tested, there are some areas that can be improved.

The application may require additional testing for invalid inputs, incorrect login credentials, duplicate user registration, and unauthorized access. The AI-related features may also require further testing to ensure that the generated responses are relevant and useful for students.

The user interface can be further improved for different screen sizes, and additional error-handling features can be added to provide clearer messages when something goes wrong.

12. Future Enhancements

AI Study Buddy can be expanded with several features to make it more useful for students.

An AI-powered study assistant can be added to answer academic questions and explain difficult concepts. The application can also include automatic note generation and summarization to help students revise their study material.

A quiz generation feature could create questions based on a student's selected subject or learning material. Personalized study plans and progress tracking can also be introduced so students can monitor their learning activities.

Other future improvements include a more advanced administrator dashboard, improved security, better mobile responsiveness, and deployment of the application so that it can be accessed online.